

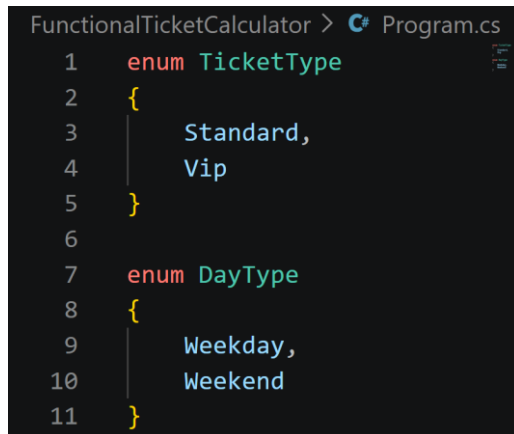
Assignment 1: Controlling Flow, Converting Types, and Representing Functions in C#

Objective: Create a console-based ticket price calculator. The program must safely convert user input, apply explicit pricing rules, represent behavior in several C# forms, and separate console operations from deterministic calculation logic.

1. Task

Implement a console application named `FunctionalTicketCalculator`.

The application receives the base price, age, student status, ticket type, and day type.



```
FunctionalTicketCalculator > Program.cs
1  enum TicketType
2  {
3      Standard,
4      Vip
5  }
6
7  enum DayType
8  {
9      Weekday,
10     Weekend
11 }
```

2. Pricing rules

- 1) A child younger than 6 receives a free ticket.
- 2) A child from 6 through 12 receives a 50% discount.
- 3) A student receives a 15% discount.
- 4) A customer aged 60 or older receives a 30% discount.
- 5) Other customers receive no customer-category discount.
- 6) A VIP ticket adds 25% after the customer discount.
- 7) A weekend visit adds 10% after the ticket-type rule.
- 8) The final price cannot be negative and must be rounded to two decimal places.

Rule priority: If several customer categories match, use the first applicable rule in the listed order. Ticket-type and day rules are then applied separately.

3. Technical requirements

3.1 Type conversion

1. `decimal.TryParse` for the base price
2. `int.TryParse` for age
3. `bool.TryParse` for student status
4. `Enum.TryParse<TEnum>` for `TicketType` and `DayType`
5. clear error messages for invalid or missing input

3.2 Control flow

1. conditional logic through if/else, switch, or conditional expressions
2. early return when input is invalid
3. at least one expression-bodied function
4. no unhandled exception for expected invalid user input

3.3 Function representation

1. at least two named static methods
2. at least two values declared with `Func<...>`
3. at least one lambda expression
4. at least one function that receives another function

3.4 Functional separation

1. calculation functions do not read from Console
2. calculation functions do not write to Console
3. no mutable global state
4. each calculation stage returns a value
5. input/output is kept in Main or another boundary method

4. Required higher-order function

The project must contain an operation with the following role. The exact name may differ.

```
13  static decimal ApplyPricingRule(  
14      decimal price,  
15      Func<decimal, decimal> rule) =>  
16      rule(price);
```

5. Test cases

PRICE	AGE	STUDENT	TICKET	DAY	EXPECTED
5000	5	false	Standard	Weekday	0.00
5000	10	false	Standard	Weekday	2500.00
5000	20	true	Standard	Weekday	4250.00
5000	65	false	Standard	Weekday	3500.00
5000	30	false	Vip	Weekday	6250.00
5000	30	false	Standard	Weekend	5500.00
5000	20	true	Vip	Weekend	5843.75

Also test negative price, empty input, text instead of age, unknown enum values, null, and zero price.

6. README questions

1. Which parts of the program are imperative?
2. Which functions are pure?

3. Where do side effects remain?
4. Why is TryParse preferred to Parse for user input?

7. Deliverables

- Complete source code as requested by the instructor
- README with run instructions and a short explanation of pure and impure parts
- At least six documented test cases
- Submission through LMS by the announced deadline
- Oral defense of the submitted code

8. Assessment policy

- Maximum score: 15 points.
- The program must compile and run during the defense.
- Defense is mandatory. A student must explain the submitted code and answer questions about any part of it.
- AI tools may be used only when explicitly permitted by the instructor and must not replace the student's understanding.
- Late submission conditions follow the official course syllabus and LMS announcement.
- Unexplained copied or generated code may receive zero according to the course academic-integrity policy.

CRITERION	FULL-CREDIT EVIDENCE	POINTS
CORRECTNESS	Application compiles; all pricing rules and priority are correct	2
TYPE CONVERSION	All required TryParse operations and invalid-input handling	2
CONTROL FLOW	Clear conditions, early exits, and predictable paths	2
FUNCTION REPRESENTATION	Methods, Func values, lambda, and higher-order function	3
FUNCTIONAL SEPARATION	Pure calculation separated from Console and global state	3
ORAL DEFENSE	Student explains decisions and predicts behavior	3
TOTAL		15